

C Programming

12_libraries

Многофайловый проект

Многофайловый проект — это подход к организации исходного кода, при котором программа разделяется на несколько отдельных файлов. С точки зрения структуры и процесса сборки, это позволяет логически изолировать компоненты, ускорить компиляцию и упростить совместную разработку.

В компилируемых языках (например, Си) многофайловость обычно реализуется через разделение на:

- Заголовочные файлы (.h) — "интерфейс" модуля. Здесь мы пишем прототипы функций и объявляем структуры, чтобы другие части программы знали, как с ними взаимодействовать.
- Файлы с исходным кодом (реализации) (.c/cpp/cc) — "логика" модуля. Здесь пишется сам код функций.

В интерпретируемых языках (например, Python) каждый файл `.py` автоматически является модулем.

- Чтобы использовать код из файла `logic.py` в файле `main.py`, мы пишем `import logic`.
- Интерпретатор находит файл, исполняет его и дает доступ к его объектам через пространство имен.

*Код ядра Linux на 2017 год содержал около 18кк (18 373 471) строк кода

Зачем?

- Один раз хорошо написанный модуль можно подключать к десяткам разных проектов
- При изменении одного файла компилятору не нужно пересобирать всю программу целиком — пересобирается только измененный модуль
- Проще ориентироваться в проекте, где все блоки разделены разложены по разным папкам и файлам по смыслу
- Если над проектом работает больше одного программиста, то в проекте с одним файлом у вас будут вечно возникать конфликты с версиями кода

```
project/  
├─ app/main.c  
└─ src/  
    │  
    │   └─ network/  
    │       │  
    │       │   └─ socket.c  
    │       │   └─ socket.h
```

```
project/  
├─ app/  
│   └─ main.c  
├─ level1/  
│   ├── src/  
│   │   └─ *.c  
│   └─ include/  
│       └─ *.h  
├─ level2/  
│   ├── level2.1/  
│   │   └─ *.c, *.h  
│   └─ level2.2/  
├─ third-party/  
├─ docs/  
├─ tests/  
└─ README.md
```

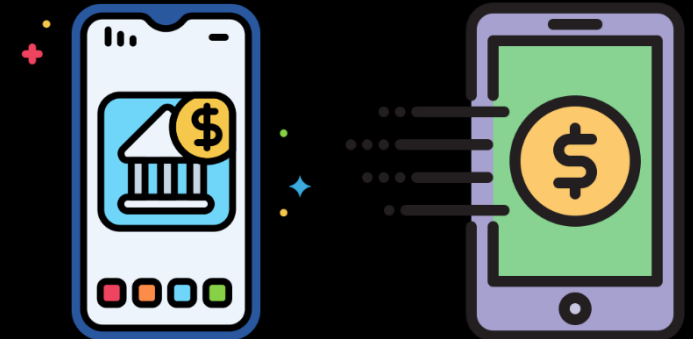
```
linux/
├── arch/                # Архитектурно-зависимый код (самый нижний уровень)
│   ├── x86/            # Код для процессоров Intel/AMD
│   └── arm/            # Код для arm процессоров
├── block/              # Работа с блочными устройствами (диски, флешки)
├── drivers/            # Драйверы устройств (видеокарты, USB, сеть)
│   ├── bluetooth/
│   └── usb/
├── fs/                 # Файловые системы (ext4, ntfs, btrfs)
├── include/            # Заголовочные файлы (.h) – интерфейсы всего ядра
├── kernel/             # Ядро ОС (мозг): управление процессами, планировщик
├── mm/                 # Memory Management (управление оперативной памятью)
├── net/                # Сетевой стек (TCP/IP, Wi-Fi и т.д.)
├── scripts/            # Вспомогательные скрипты для сборки ядра
└── Makefile            # Главный файл, управляющий сборкой этого гиганта
```

```
SuperMegaGame/
├── assets/                # Графика, звуки, конфиги (JSON/YAML)
│   ├── sprites/
│   ├── levels/
│   └── sounds/
├── src/                  # Исходный код
│   ├── main.py           # Точка входа: инициализация и запуск цикла
│   ├── engine/          # Ядро (базовые классы)
│   │   ├── camera.py
│   │   ├── display.py   # Настройка окна и событий
│   │   └── input.py     # Обработка клавиатуры/мыши
│   ├── entities/        # Игровые объекты
│   │   ├── base.py      # Базовый класс Entity
│   │   ├── player.py    # Класс игрока
│   │   └── enemy.py     # Логика врагов
│   ├── scenes/          # Уровни и состояния игры
│   │   ├── menu.py      # Код главного меню
│   │   └── level_1.py   # Логика первого уровня
│   └── utils/           # Вспомогательный код
│       ├── helpers.py   # Математика, загрузчики ресурсов
│       └── constants.py # Настройки: FPS, цвета, пути к файлам
├── tests/               # Тесты (pytest)
├── requirements.txt     # Список библиотек (pygame, numpy и т.д.)
└── README.md
```

Как разделять?

- Один модуль - одна задача, если файл работает с сетью (network.c), то ему не нужно отрисовывать какой-нибудь графический интерфейс
- По возможности стоит делать так, чтобы модули значили как можно меньше друг о друге, тогда сохранится минимальная зависимость и такие модули легко удалять/добавлять в проекты
- Соблюдать иерархию: модули высокоуровневые могут зависеть от низкоуровневых, но не наоборот. Например в core/log.c есть реализация логирования, что будет нижним уровнем, а верхний уровень это тот кто использует логику core/*
- Нет смысла разделять проект на 100 файлов если всего в нем 200 строк кода, такая избыточность ни к чему хорошему не приведет
- DRY (Don't Repeat Yourself): Если копируете один и тот же код в разные файлы, то значит, что его пора вынести в отдельный модуль

*Выделить модули или задачи в проекте - навык, что обычно приходит с практикой



Сборка

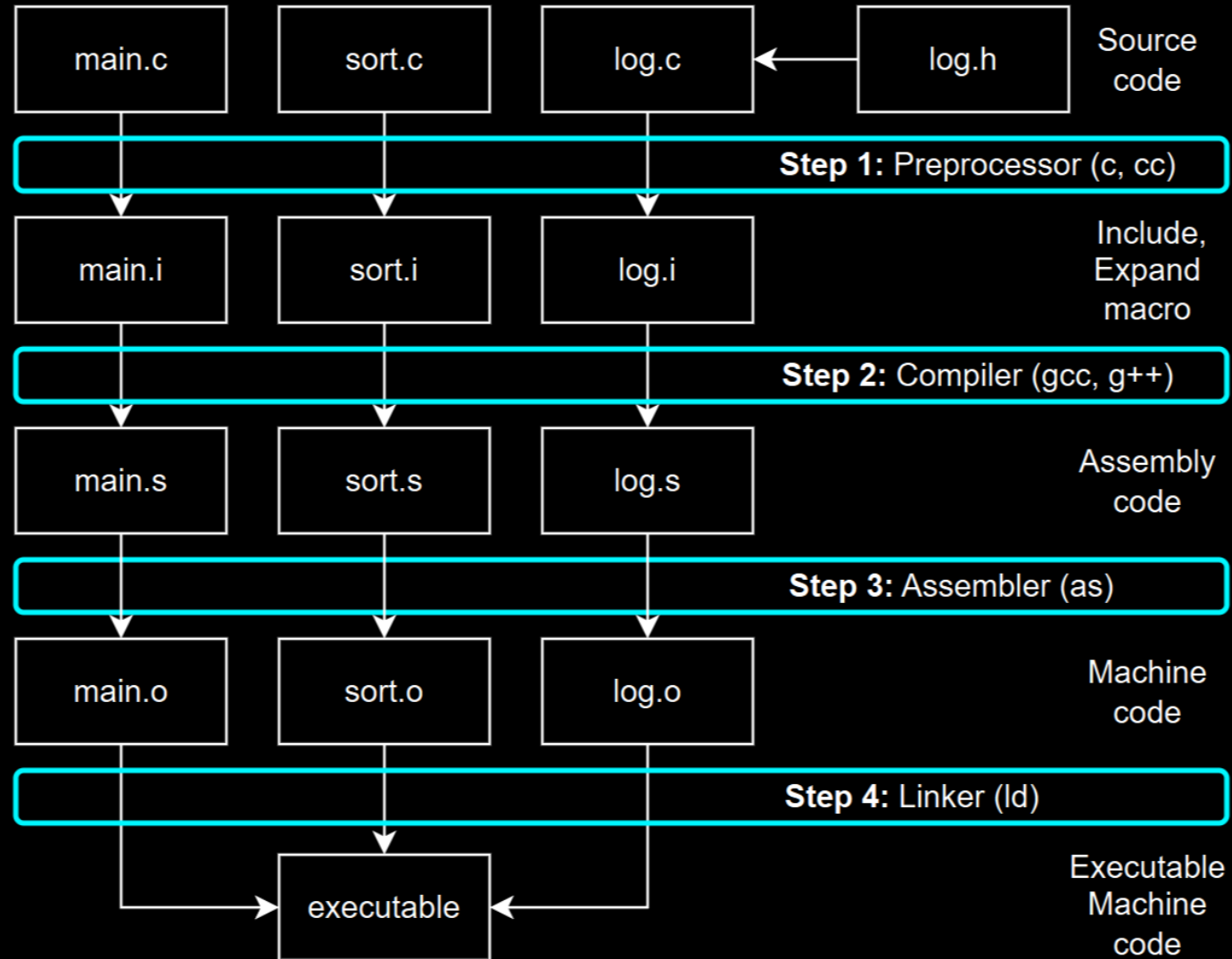
```
project/  
├── app/main.c  
└── src/  
    ├── network/  
    │   ├── socket.c  
    │   └── socket.h
```

```
#include "src/network/socket.h"
```

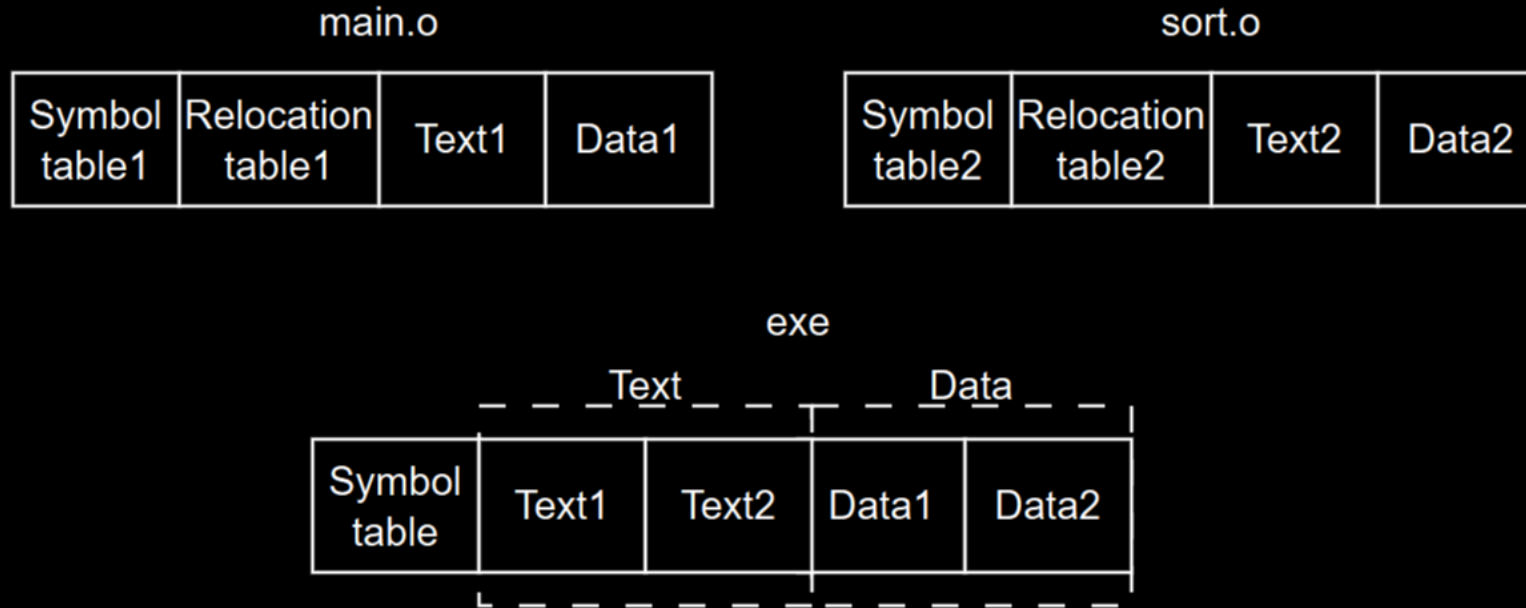
```
#include "socket.h"
```

```
gcc ./app/main.c ./src/network/socket.c -I./src/network/ -o app
```

Compilation



Object files



Объектный файл состоит из:

- таблицы символов - таблица с именами всех глобальных переменных и функций
- таблицы релокаций - адреса (указатели) на заглушки в коде куда нужно подставить адрес вызова undefined функции когда мы найдем такую в других таблицах символов
- секции text - код самих функций
- секции data - глобальные (и static) инициализированные переменные

*Команда в Linux `nm`, пример `nm main.o`. D - секция Data, U - undefined, T - секция text

Linker errors

- Если функции нет в sort.c, тогда линковщик выдаст ошибку **undefined reference to 'sort'** и завершит свою работу, т.к. не смог найти код функции sort()
- Если определение функции или две глобальных переменных с одним именем встречаются в нескольких объектных файлах, то линковщик также выдаст сообщение об ошибке: **multiple definition of 'sort'** и завершит работу с ошибкой

Library (Библиотека)

Библиотека (от англ. library) - это сборник готового кода (функций, объектов), который мы подключаем к своей программе, чтобы не изобретать велосипед.

Для интерпретируемых языков (python например) библиотека — файл, содержащий либо код на исходном языке (.py), либо байт-код для виртуальной машины. Пример библиотеки для Python — Tkinter

Пример библиотеки компилируемых языков (Си например): стандартная Си библиотека с функцией printf() — LibC. Реализация (логика) в библиотеке, интерфейс в заголовочном файле.

Стандартные пути в Linux:
/usr/include и /usr/lib/



Static library

Статическая библиотека - набор объектных файлов (архив) содержащий код функций. Стандартные библиотеки многих компилируемых языков программирования распространяются в виде объектных файлов

main.o

Symbol table1	Relocation table1	Text1	Data1
---------------	-------------------	-------	-------

libc.a

Index	Symbol table2	Relocation table2	Text2	Data2	Symbol table3	Relocation table3	Text3	Data3
printf.o					scanf.o			

exe

Text		Data		
Symbol table	Text1	Text2	Data1	Data2

+

Все функции включаются в один исполняемый файл, который легко перемещается

-

Исполняемый файл большой по весу.
Дублирование кода.
Ошибка в библиотеке - пересборка всей программы

Linux/Mac: .a
Windows: .lib



Dynamic library

Динамическая библиотека - исполняемый файл, содержащий машинный код. Код функций из такой библиотеки не встраивается в наш код. Является исполняемым файлом и загружается в память загрузчиком программ ОС либо при запуске программы, либо по запросу уже работающей программы

+

- Экономия памяти за счёт использования одной библиотеки несколькими процессами
- Возможность исправления ошибок (достаточно заменить файл библиотеки и перезапустить работающие программы) без изменения кода основной программы
- Подключение на горячую (плагин)

-

- Возможность нарушения API — при внесении изменений в библиотеку существующие программы могут перестать работать (утратят совместимость по API);
- Конфликт версий динамических библиотек, — разные программы могут нуждаться в разных версиях библиотеки (API)
- Злоумышленник вместо вашей библиотеки может подсунуть вам свою библиотеку со своим кодом

Linux: .so (shared object)

Mac: .dylib (dynamic library)

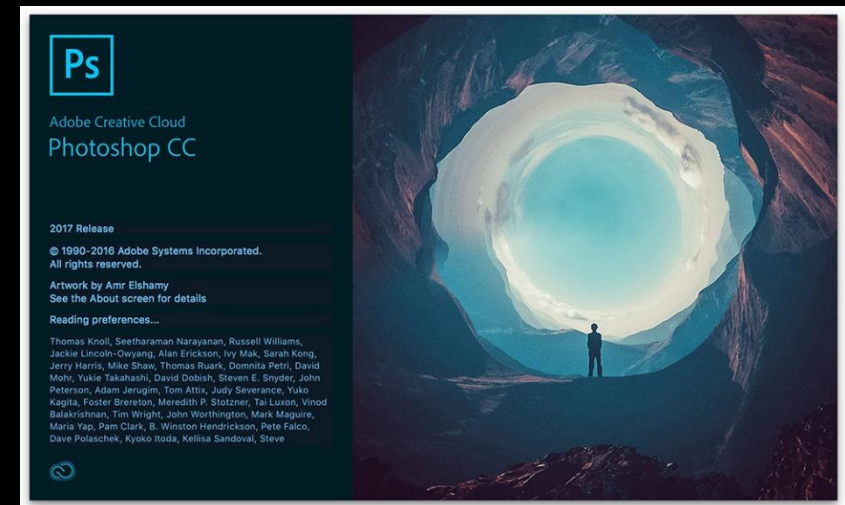
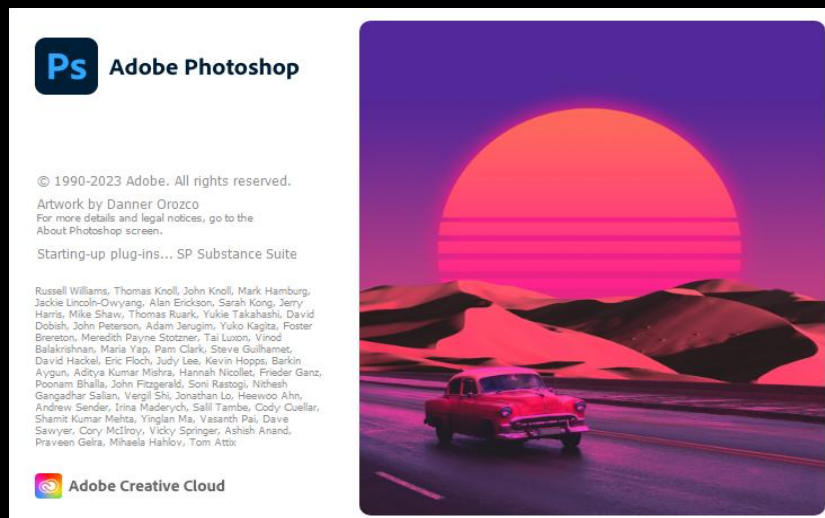
Windows: .dll (dynamic link library)



Plug-in

Плагин - дин. библиотека, которую разработчик специально оформил для возможности загружать и исполнять чужой код.

- Разработчики фотошопа описывают правила по которым можно подключить библиотеку и вызывать из нее функции.
- Любой программист, не имея доступа к исходному коду фотошопа, пишет свою библиотеку (.dll или .so) с новой кистью или фильтром
- Человек кладет файл библиотеки в папку plugins. При запуске Photoshop сканирует эту папку и подтягивает новый код в себя.



Dynamic linking

Динамическая линковка происходит во время выполнения программы:

- На этапе компиляции к исполняемому файлу записываются ссылки на динамическую библиотеку (имя/путь). Когда вы запускаете программу, загрузчик ОС (динамический линкер) находит эти файлы в системе, загружает их в память и связывает с вашей программой
- Программа запускается и уже в процессе работы сама решает загрузить библиотеку (dlopen() (в Linux) или LoadLibrary() (в Windows)). Именно так работают плагины

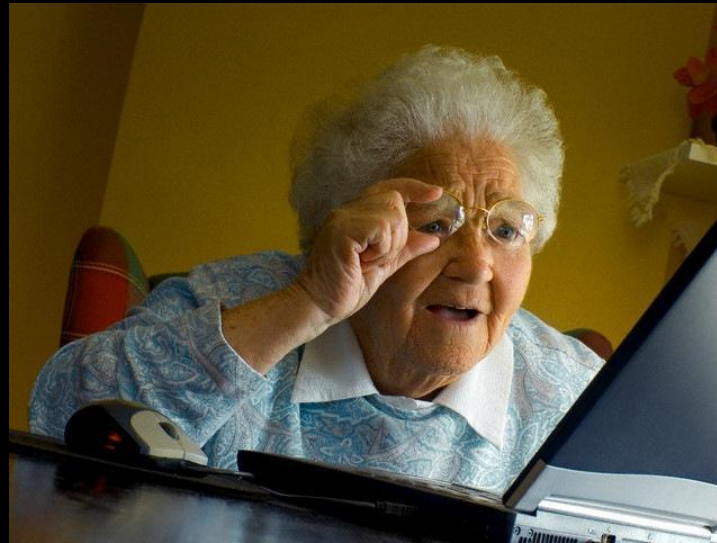
```
#include <dlfcn.h>
```

```
void *dlopen(const char *filename, int flags);  
int dlclose(void *handle);  
char *dlerror(void);  
void *dlsym(void *handle, const char *symbol);
```

*Кому интересно системное программирование и как происходит линковка динамических библиотек - идем читать в интернет или в книжках про PIC (Position-independent code) и таблицы GOT, PLT с интересными примерами на ассемблере.

Библиотеки

Библиотеки имеют свои плюсы и минусы и нужно отталкиваться как и всегда от конкретных целей и задач



Build Static Lib

```
gcc app.c -c
```

```
gcc log.c -c
```

```
ar rc libMY_LOG.a log.o
```

```
gcc app.o -o static_exe -L.
```

```
-lMY_LOG
```

```
./static_exe
```

*Посмотреть таблицу символов – команда nm

Build Dynamic Lib

```
gcc app.c -c
```

```
gcc log.c -c -fPIC
```

```
gcc --shared app.o log.o -o libMY_LOG.so
```

ИЛИ

```
gcc app.o -o dynamic_exe -L. -lMY_LOG -Wl,-rpath,.  
./dynamic_exe
```

```
gcc app.o -o dynamic_exe -L. -lMY_LOG  
LD_LIBRARY_PATH=. ./dynamic_exe
```

*Посмотреть список дин. либ – команда ldd

*Предзагрузить либу: LD_PRELOAD

Linux library version

- Major (Мажорная): 4.x.x — Критическое изменение API. Старый код не запустится с новой версией.
- Minor (Минорная): x.2.x — Добавление новых функций. Старый код будет работать.
- Patch (Патч): x.x.3 — Исправление багов внутри функций. API не меняется.

Такое версионирование еще называется семантическим:
SemVer (Semantic Versioning)

```
ls /usr/lib/x86_64-linux-gnu/
```

<https://github.com/kruffka/C-Programming>

